PRODUCT + TECHNICAL BLUEPRINT

An offline Android app that turns user-recorded sounds into playable beat loops

BLUEPRINT STATUS

Document version	1.0 - confirmed decision baseline
Prepared on	13 October 2026
App name	Undecided - intentionally not invented
Application ID / package name	Undecided - intentionally not invented
Minimum Android version	Android 12 or newer
Primary implementation language	Kotlin
Distribution intent	Project demonstration + preparation for possible Google Play release

How to read this document

Confirmed choices are recorded as requirements. Open items are labelled as undecided rather than guessed. Technical recommendations are identified as proposals and must be validated during implementation; this document does not claim that the app has already been implemented or tested.

Working title only: “Android Sound-to-Beat App” is a descriptive document title, not a chosen product name.

Document map

- 1. Product definition and design principles
- 2. Scope and boundaries
- 3. End-to-end user journey
- 4. Functional requirements
- 5. Sound DNA and transformation system
- 6. Beat editor, loops, and sound pads
- 7. Projects, local storage, import, and export
- 8. Technology stack and architecture
- 9. Audio-engine pipeline and technical constraints
- 10. Interface and visual behavior
- 11. Privacy, permissions, and platform behavior
- 12. Quality attributes, risks, and mitigations
- 13. Verification and acceptance checklist
- 14. Implementation roadmap
- 15. Open decisions register
- 16. Release-readiness checklist and references

Decision status key

Label	Meaning
CONFIRMED	Explicitly chosen during ideation; treat as a product requirement.
PROPOSED	A technical implementation recommendation based on the agreed product; validate before locking it.
OPEN	Not decided. Do not fill in a value or behavior without a later decision.

Executive snapshot

The app accepts one or more user-supplied sound sources. Each source is limited to 10 seconds after capture/selection. An offline Sound DNA analysis presents six chosen technical measurements plus intuitive sound-personality descriptions and visualizations. The app suggests an adaptive number of transformations across five modes, previews the transformed sound first, and then generates a beat-loop preview for a chosen mode. Users edit that loop with a configurable step sequencer and playable pads, save editable projects locally, and export a mixed loop or individual layers.

Hard product boundary

Every musical layer must be derived from the user-provided audio. The MVP does not use independent synthesized instruments, bundled sample packs, cloud AI, or external API keys. The app is offline-first by requirement; all core functions must work without internet.

1. Product definition and design principles

Product statement

A native Android creative tool that lets users record everyday sounds or import local audio, inspect a sound fingerprint, explore musical transformations, and build repeating beat loops from those sounds. The app should be playful to explore but produce a real, editable audio result.

Product principles

- Let the sound guide the creative suggestions; do not require users to choose a genre before hearing possibilities.
- Keep the raw source recognizable in some transformations while allowing radical transformation in others. Recognizability is a user-adjustable creative outcome; detailed control UI remains to be designed.
- Use one creative mode per current beat. Switching modes changes the sound transformations while carrying over compatible current settings.
- Build from the user's source audio only. Repetition, slicing, reversing, filtering, pitch changes, time manipulation, granular transformation, and other selected effects may create derived layers; independent synth or bundled samples are out of scope.
- Keep all recording, analysis, transformation, editing, saving, and export workflows local and usable without network access.
- Keep music-production complexity behind a focused interface: visual analysis and playful previews first, then a sequencer and pads for editing.
- Distinguish confirmed requirements from open implementation choices throughout development.

Primary user outcome

The user can take a mundane sound - for example, a clang, knock, scrape, rattle, or hum - and transform it into a beat loop they can customize, perform with, save, and share.

Non-goals for the MVP

- Generating a full multi-section song arrangement; the output scope is repeating beat loops.
- Cloud accounts, online processing, cloud backup, online sharing services, or external AI APIs.
- Using independent synthesized instruments or a bundled sound/sample library to fill gaps in the source audio.
- Requiring multiple creative modes to be blended into one beat. The current beat uses one selected mode, with mode switching supported.
- Background beat playback. Playback is restricted to the foreground and stops when the app leaves the foreground or the phone locks.
- Assuming a final app name, package name, target SDK, compile SDK, exact tempo defaults, codec implementation, or numerical resource limits.

2. Scope and boundaries

Included in MVP scope

Area	Confirmed in-scope behavior
Input	Record using the microphone; import local audio files; support isolated events and longer clips as inputs, with each selected source segment limited to 10 seconds.
Sound analysis	Display intuitive sound-personality descriptions and six chosen technical measurements with visualizations.
Discovery	Offer five modes - Industrial, Rhythmic, Melodic, Ambient, Glitch - with an adaptive number of initial suggestions and on-demand generation for remaining modes.

Area	Confirmed in-scope behavior
Audio transforms	Sample slicing; pitch shifting; playback speed and stretching; filtering; distortion and bit reduction; delay and echo; reverb; reverse playback; retrigger and stutter; granular transformation.
Beat editing	Tempo, rhythm pattern, beat density, pitch, effect intensity, layer volume, loop length, timing/swing, and manual sound placement.
Performance	A configurable step-sequencer grid and automatically populated, user-reassignable sound pads.
Projects	Autosave; reopen editable project; retain rendered audio; view, open, rename, and delete projects.
Output	Export full mix as WAV or MP3; export individual sound layers as WAV; save locally or share with Android's system share UI.
Platform	Native Android, Kotlin, Android Studio, modern Android phones, Android 12+ minimum, portrait and landscape adaptive layouts.
Distribution	Support project demonstration and prepare for possible public Google Play submission; no approval is assumed.

Important boundaries

Topic	Locked boundary
Source duration	Maximum 10 seconds for each source input. Microphone capture warns near the limit and stops automatically at 10 seconds. Longer imports require the user to select a section no longer than 10 seconds.
Generated loop duration	May exceed 10 seconds because transformed source segments can be repeated and arranged. Output duration limit is open.
Multi-source projects	A project may contain one or multiple source recordings/imports. No maximum source count is specified. The user selects or accepts a recommended primary source.
Creative mode	One selected mode applies to the beat. Cross-mode layer blending is not required. Mode switching carries over compatible settings while changing transformations.
Internet	All core features must work offline; project data stays on device unless the user explicitly exports or shares.
Audio formats	Import format goal: WAV, MP3, M4A/AAC, and other practical formats where feasible. Actual decoder compatibility must be tested; it is not yet verified.

3. End-to-end user journey

The flow below reflects the selected interaction model. Screen labels are conceptual navigation labels; exact screen designs, colors, fonts, and navigation chrome are not yet chosen.

Step	Stage	Required behavior
01	First launch	A short interactive walkthrough introduces the premise and guides the user through a first recording and transformation. Tutorial length and exact screens are open.
02	Choose source	Record with the microphone or import a local audio file. Longer imported audio is trimmed by the user to a selected section of 10 seconds or less.
03	Capture / trim review	For microphone recording, show a warning/countdown near 10 seconds and stop automatically at the limit. If audio appears unsuitable, warn and offer re-record or continue; thresholds are open.
04	Sound DNA	Analyze the source locally. Show the six selected measurements, intuitive sound-personality descriptions, waveform/frequency visualizations, and reactive visual treatment.
05	Explore mode suggestions	Suggest an adaptive number of sound transformation previews based on Sound DNA. The previews demonstrate transformed source audio, not full beat loops. Remaining modes are generated on demand.
06	Select a creative mode	The five modes are Industrial, Rhythmic, Melodic, Ambient, and Glitch. One mode applies to the current beat. In multi-source projects, a primary source drives the interpretation and the other sources contribute under the same mode.
07	Beat-loop preview	After the user selects a mode, generate a repeating beat-loop preview for that mode before opening the editor.
08	Edit and perform	Use the configurable step sequencer and playable sound pads. Customize the nine confirmed controls. Simple tempo/volume updates may apply during playback when feasible; changes requiring regeneration apply when stopped or restarted.
09	Multi-source changes	Users can add/remove sources while editing. When a source is added or removed, ask whether to update the beat or preserve the current arrangement. Regeneration timing/mechanics remain open.
10	Autosave and manage	Automatically save project changes as users work. In the library, users can view/open, rename, and delete projects. Preserve editable data and finished renders.
11	Export / share	Export complete mixed beat as WAV or MP3, or individual layers as WAV. Let the user save to the device or share via Android's system sharing interface.

4. Functional requirements

4.1 Recording and import

ID	Requirement	Verification intent
FR-01	Record a sound from the device microphone.	Grant/deny microphone permission; record, stop, and replay on a real device.
FR-02	Warn near the 10-second capture limit and auto-stop at 10 seconds.	Recording stops at the cap; user can review captured audio.
FR-03	Import local audio in WAV, MP3, M4A/AAC, and other practical formats where feasible.	Verify each claimed format on supported Android 12+ devices; unsupported formats fail clearly.
FR-04	If imported audio exceeds 10 seconds, let the user select a segment no longer than 10 seconds.	Confirm selected range obeys the cap.
FR-05	Support both isolated sound events and short clips.	Single impact and multi-event clips are accepted within per-source cap.
FR-06	If an input appears unsuitable, warn and allow re-record or continue.	Both choices are available; do not silently block continuation.
FR-07	Support one or more sources in a project; do not invent a source-count limit.	Create a multi-source project without enforcing an arbitrary documented limit.

4.2 Sound DNA and transformations

ID	Requirement	Verification intent
FR-08	Analyze and display RMS energy, transient strength, spectral brightness/centroid, dominant frequency, decay time, and spectral flux.	All six selected metrics are available when analysis succeeds; no unapproved extra metrics are required.
FR-09	Combine intuitive sound-personality descriptions with technical measurements and visualizations.	Analysis includes the descriptive and technical views.
FR-10	Use Sound DNA to drive creative suggestions and primary-source recommendations.	Suggestions/recommendations respond to audio features; exact scoring and thresholds are still open.
FR-11	Offer Industrial, Rhythmic, Melodic, Ambient, and Glitch modes.	All five modes are reachable.
FR-12	Generate an adaptive number of initial transformed-sound previews; generate remaining modes on demand.	Initial count is not fixed in this specification; behavior must adapt to Sound DNA.
FR-13	After selecting a mode, generate a beat-loop preview before opening the editor.	Sequence is transformed-sound preview -> mode choice -> beat-loop preview -> editor.
FR-14	Implement the ten selected transformations.	Each effect is tested for correct execution, stability, and audibility.

4.3 Beat editing and pads

ID	Requirement	Verification intent
FR-15	Provide step-sequencer grid + playable sound pads using the same user-provided audio material.	Both editing/performance surfaces operate on derived source audio.
FR-16	Allow configurable number of grid steps and loop length.	Grid is not hard-coded to a fixed count; safe limits must be decided during implementation.

ID	Requirement	Verification intent
FR-17	Expose tempo, rhythm pattern, beat density, pitch, effect intensity, layer volume, loop length, timing/swing, and manual sound placement.	All nine customization categories are represented.
FR-18	Estimate starting tempo when a repeating pulse is detectable; otherwise use a changeable default BPM.	Pulse-present and pulse-absent cases behave as specified; numerical default remains open.
FR-19	Auto-populate pads from available source segments/transformations; let users reassign pads.	Pads are populated, and reassignment works.
FR-20	Choose initial pad count based on available sound segments; let user change it.	Count changes with available material and can be manually changed; limits/defaults remain open.
FR-21	Carry over compatible settings when switching modes, while changing sound transformations.	Settings remain where compatible; no per-mode edit history is implied.
FR-22	When sources are added/removed, ask whether to update the beat or preserve the current arrangement.	Both choices are presented; exact regeneration behavior is open.
FR-23	Allow generated loops longer than 10 seconds.	Source cap is not incorrectly applied to rendered loop duration.

4.4 Projects and export

ID	Requirement	Verification intent
FR-24	Autosave project changes while the user works.	Make edits, leave/reopen project, verify changes persist. Exact autosave timing is open.
FR-25	Save editable project data and finished audio renders.	Reopen and continue editing; saved render is playable.
FR-26	Library supports viewing/opening, renaming, and deleting projects.	All three operations work against local projects.
FR-27	Export full mixed loop as WAV or MP3.	Each format opens/plays in independent players; no bitrate/sample rate promise.
FR-28	Export individual sound layers as WAV only.	Each chosen layer produces a WAV file.
FR-29	Save export to a user-selected device location or invoke Android system sharing.	Both routes work without the app requiring a cloud account.
FR-30	Stop playback when app leaves foreground or phone locks.	Background/lock transition stops playback.

5. Sound DNA and transformation system

Sound DNA measurements (confirmed)

Measurement	Purpose in the concept	Specification status
Energy (RMS)	Describe signal energy over time.	Measurement selected; exact windowing/normalization is open.
Transient strength	Describe the sharpness of a sound onset.	Measurement selected; detector definition/thresholds are open.
Spectral brightness (centroid)	Describe relative concentration of high-frequency energy.	Measurement selected; calculation parameters are open.
Dominant frequency	Highlight a prominent frequency component.	Measurement selected; estimator and edge cases are open.
Decay time	Describe how quickly the sound fades.	Measurement selected; precise decay definition is open.
Spectral flux	Describe change in spectral content over time.	Measurement selected; window size and thresholds are open.

The app must pair these technical displays with intuitive sound-personality descriptions. Example words such as “punchy”, “bright”, “metallic”, “resonant”, or “noisy” were discussed illustratively, but the final label vocabulary is not fixed.

Creative modes (confirmed)

Mode	Product intent	Implementation note
Industrial	Emphasize hard, metallic, abrasive or mechanical character.	Creative direction, not a locked parameter preset.
Rhythmic	Emphasize groove, repetition and percussive patterns.	Creative direction, not a locked parameter preset.
Melodic	Explore pitched and tonal interpretations of source audio.	No synthesized notes/instruments may be added independently of source audio.
Ambient	Explore softer, spacious, sustained or atmospheric interpretations.	Must remain derived from source audio using selected transforms.
Glitch	Explore granular, chopped, retriggered or fragmented rhythm.	Granular transformation and retrigger/stutter are selected effects.

Selected transformation set (all ten)

- Sample slicing - isolate or split sections of a source recording.
- Pitch shifting - alter perceived pitch of source-derived audio.
- Playback speed and stretching - alter playback rate and/or duration.
- Filtering - change frequency emphasis.
- Distortion and bit reduction - add roughness or reduced-resolution character.
- Delay and echo - repeat source-derived audio over time.
- Reverb - add a sense of space to source-derived audio.
- Reverse playback - reverse a source or selected segment.
- Retrigger and stutter - repeat short sections to create rhythmic patterns.
- Granular transformation - break audio into small fragments and rearrange or layer them.

Core transformation rules

- No cloud model, generative AI API, or external API key is required or included in the plan.

- No independently synthesized instruments or bundled sample library may provide musical layers in the MVP.
- All layers must be traceable to one or more project source recordings/imports. A single source can be sliced, transformed, repeated, layered, and arranged.
- Recognizability is adjustable in the creative result: some transformations preserve the source character; others may radically alter it.
- One mode is active for a beat at a time. Changing modes changes the transformation interpretation and preserves only compatible settings.
- The initial number of transformed-sound previews is adaptive to Sound DNA. A user chooses a mode; only then is the beat-loop preview generated for that mode. Remaining mode previews are generated on demand.

Feasibility note

The requirement is to make each selected transformation work locally; the quality and performance of advanced effects such as granular transformation, pitch shifting and time stretching must be tested on physical Android devices. This blueprint does not claim the effects have already been implemented or benchmarked.

6. Beat editor, loops, and sound pads

Step sequencer

The editor combines a step-sequencer grid with playable sound pads. The grid uses transformed material from the project sources; it is not constrained to a confirmed fixed number of steps. The number of steps and loop length are user-configurable.

Confirmed behavior	Still open
Configurable number of steps and loop length.	Safe minimum/maximum step count; allowed loop lengths; any time-signature choices.
Manual sound placement.	Exact placement gestures and grid visual design.
Tempo control; adaptive initial BPM estimation.	Default BPM, permitted BPM range, pulse-detection method and confidence threshold.
Timing/swing control is included.	Swing algorithm, its units/granularity and how it interacts with step count.
Loop may be longer than 10 seconds.	Maximum render duration, output file size guardrails and supported length combinations.
Simple controls such as volume and tempo may update during playback when technically feasible; changes requiring regeneration apply after stop/restart.	Which precise controls can update live on supported devices; no real-time guarantee applies to every effect.

Sound pads

- The app initially selects pad count based on available source segments; user can change the count.
- Pads are automatically populated and can be reassigned to other segments or transformations from the user-provided source material.
- Pad count limits, default count, grid/layout, hit feedback and simultaneous-trigger behavior have not been selected.
- The same source material powers pad performance and step-sequencer playback.
- Pad behavior in multi-source projects beyond the agreed source-driven concept is an open design detail; do not invent a rule for source assignment.

Mode and source behavior

Situation	Confirmed response
Switch mode	Carry over compatible current settings while replacing the sound transformations. Do not assume mode-specific history or undo/redo.
Add/remove source	Ask user to update the beat or preserve current arrangement. How the choice is implemented and when regeneration occurs remains open.
Choose primary source	App recommends a primary based on Sound DNA; user can accept or select another. Recommendation metrics are open.
Additional sources	Contribute transformed material under the same selected creative mode as the primary source.
Multiple mode layers	Cross-mode blending is not a requirement; the current beat uses one selected mode.

7. Projects, local storage, import, and export

Project persistence

Each project must preserve editable project data and a finished audio render. Project changes autosave as users work. Users can later reopen an editable project, and the local library supports viewing/opening, renaming, and deleting projects.

Conceptual project data (logical model; schema not locked)

Record type	Information required to support confirmed behavior	Status
Project	Project identifier; user-visible project name; references to source items, selected mode, editable arrangement/settings, and rendered output.	Logical proposal; exact schema/open ID format undecided.
Source item	Source type (microphone/import), local storage reference, trimmed segment reference if applicable, Sound DNA measurements, and role as primary/additional source.	Fields proposed from requirements; exact persistence/analysis cache strategy is open.
Arrangement	Sequencer configuration, pad assignments, layer placement, selected mode, and nine confirmed edit control values.	Required data concepts; exact serialization/schema is open.
Rendered output	Reference to the mixed loop render; references to individual layer renders when exported/stored.	Required behavior; file naming, format metadata and retention details are open.
Autosave state	Persist edits automatically while user works.	Autosave interval/trigger and crash recovery strategy are open.

Storage recommendation (proposed)

Keep original and rendered audio in app-specific local storage and maintain project metadata locally. Room/SQLite is a reasonable candidate for the metadata index, but Room is not a confirmed dependency. A simpler local metadata store could be considered during implementation. No account, cloud backup, server database or online sync is in scope.

Import behavior

- Use Android's system file picker so a user explicitly selects a local audio file; a permission to read the entire media library is not assumed by this blueprint.
- Intended file formats: WAV, MP3, M4A/AAC and other practical formats where feasible. The supported list must be verified on target devices after selecting a decoding path.
- If a selected file exceeds 10 seconds, the user must choose a segment no longer than 10 seconds for analysis and beat creation.
- Every source entry in a multi-source project is subject to the per-source 10-second cap. The number of sources has no specified maximum in this blueprint.

Export and sharing

Export item	Allowed output	Delivery
Complete mixed beat loop	WAV or MP3	User can save to a chosen device location or share through Android's system sharing interface.
Individual sound layer	WAV only	User can save to a chosen device location or share through Android's system sharing interface.
Editable project	Local project data + finished render retained in the app	No cloud export/sync requirement.

Encoder/decoder validation

MP3 export requires an offline encoding solution. The specific encoder, licensing, performance and device compatibility are not finalized. Do not assume that every Android device exposes the same usable encoder. Verify this before treating MP3 export as complete.

8. Technology stack and architecture

Chosen application stack

Layer	Baseline	Status / note
IDE / build environment	Android Studio	CONFIRMED at product level; exact IDE/Gradle/plugin versions remain open.
Language	Kotlin only for the MVP	CONFIRMED. Java, Python and C++ are not required in the v1 app stack.
UI	Jetpack Compose	Chosen implementation baseline from the discussion; official Android guidance positions Compose as its modern native UI toolkit [1][2].
Microphone capture	Android AudioRecord / PCM capture	Chosen technical baseline; test configuration and device behavior on real phones [3].
PCM playback	Android AudioTrack	Chosen technical baseline for supplying/playing audio buffers [4].
Audio processing	Custom offline DSP engine	Chosen approach at concept level; no cloud AI or external API. Implementation details/algorithms remain to be designed.
Concurrency	Background processing outside the UI thread; Kotlin coroutines/ViewModel are candidates	PROPOSED implementation detail; exact dependencies not locked.
Local metadata	Local storage; Room/SQLite is optional candidate	PROJECT REQUIREMENT is local persistence; Room itself is not confirmed.

Layer	Baseline	Status / note
Audio import	Android Storage Access Framework/system file picker	PROPOSED path for user-selected files; official SAF docs describe user-mediated file access [5].
Audio export	WAV writer; offline MP3 encoder to be selected	Formats are confirmed; encoder implementation is OPEN and must be verified.
Share flow	Android system sharing interface	CONFIRMED behavior; exact intent/file-provider implementation is technical work.
Network/backend	None required for core use	CONFIRMED full offline operation; no online accounts or API keys in scope.

Architecture boundaries

Component	Responsibility
Presentation layer	Jetpack Compose screens, visualizations, adaptive layout and user controls.
View/state layer	Coordinate recording, analysis, previews, editor state, autosave and export state. Keep long-running work off the UI thread.
Audio I/O layer	Microphone capture; source decode; PCM handling; playback buffers; WAV output; candidate MP3 encoding path.
Sound DNA analyzer	Compute the six selected measurements and prepare the basis for intuitive descriptors/suggestions.
Transformation engine	Implement the ten selected DSP transformations and mode-specific parameter sets, locally.
Beat engine	Build source-derived layers, schedule rhythmic events, mix/loop the render, and create individual layer renders.
Project repository	Persist project metadata, source references, editable arrangement and audio renders locally.
Export/share adapter	Write chosen export format to user-selected destination or route to Android sharing UI.

Module/file structure (proposed, not final)

A maintainable implementation could separate packages for ui, recording, import, analysis, dsp, modes, sequencer, pads, playback, projects, export, and platform integration. These are logical boundaries rather than finalized package names. The app's final namespace/application ID is intentionally open.

9. Audio-engine pipeline and technical constraints

Processing sequence

Stage	Behavior
1. Capture/import	Obtain a recording or user-selected audio file. Enforce the per-source 10-second selection rule.
2. Decode/normalize	Decode selected input into an internal PCM representation that supports analysis and effects. Sample rate, channels, bit depth and normalization policy remain open.
3. Inspect quality	Assess whether the source appears unsuitable. If so, warn and offer re-record/continue; threshold design remains open.
4. Analyze Sound DNA	Calculate RMS energy, transient strength, spectral centroid, dominant frequency, decay time, and spectral flux.

Stage	Behavior
5. Derive suggestions	Use the primary source's Sound DNA to recommend mode previews and, in multi-source projects, a primary source candidate. Scoring and exact number of first suggestions remain open.
6. Transform source	Apply one selected mode's applicable transforms. All output audio must remain derived from source input.
7. Preview transformed sound	Play a transformed-source preview before a beat loop has been chosen.
8. Build loop preview	After mode selection, arrange transformed source segments into a repeating beat loop for the chosen mode.
9. Edit and render	Edit tempo/pattern/density/pitch/effects/volume/length/swing/manual placement using the grid and pads. Render or regenerate when required.
10. Save/export	Autosave editable project + finished render. Export the mix as WAV/MP3 or individual layers as WAV.

Audio constraints and engineering notes

- Audio quality depends on the source and selected transformations. Because no independent synth or library is allowed, the engine cannot guarantee a convincing kick, bass line, or melody from every recording.
- Single impact sounds do not necessarily contain a reliable pulse. When a repeating pulse is detectable, estimate an initial tempo; otherwise choose a configurable default BPM whose numeric value is still open.
- Pitch shifting and time stretching are distinct concepts; quality-preserving algorithms may be more expensive than simple playback-rate changes. The chosen approach must be tested on devices.
- Granular transformation can increase CPU/memory cost. The implementation must manage processing off the UI thread, avoid blocking audio playback, and gracefully handle failure or memory pressure.
- Audio output can vary by device, microphone processing and codec support. The code must check supported recording configuration and handle unsupported/failed setup gracefully.
- Output loop duration limits, grid limits, safe layer counts, simultaneous pad playback and performance targets are open and should be set after device testing.
- The app's foreground-only playback rule simplifies lifecycle scope: stop active playback on foreground loss or screen lock. Recording remains an explicitly user-initiated foreground action.

Possible implementation refinements (not confirmed)

- Use Kotlin coroutines or another appropriate scheduling approach for analysis/render work away from the main thread.
- Prototype algorithm choices with short source clips and compare quality/performance on physical Android devices before selecting defaults.
- If AudioTrack performance is insufficient for pad responsiveness, evaluate lower-latency audio options later; C++/Oboe is not part of the initial stack commitment.
- Choose an offline MP3 encoder only after checking compatibility, licensing, build size and maintenance. No particular encoder/library is selected.

10. Interface and visual behavior

Visual identity

The interface has an adaptive visual identity that responds to the recorded sound while maintaining a consistent layout. The app includes accurate audio visualizations and expressive reactive animations. The palette, typography, icon style and precise visual language are not chosen.

Confirmed visual/content behavior

Area	Requirement	Open details
Sound display	Waveform / frequency-oriented visualization and expressive animations that react to transformation/playback.	Specific chart type, animation timing and colors.
Sound DNA	Technical measurements plus intuitive personality descriptions.	Final descriptor vocabulary, labels, thresholds and layout.
Mode discovery	An adaptive number of initial source-transform previews, then remaining modes on demand.	Exact adaptive-count logic, preview ordering and loading treatment.
Editor	Step grid + sound pads; controls for nine selected customization categories.	Grid dimensions, control placement, limits and interaction gestures.
Orientation	Portrait and landscape, with adaptive layout rearrangement.	Screen-specific arrangements are not finalized; tablet support is not assumed.
Onboarding	Short interactive first-launch walkthrough, guiding first recording/transformation.	Tutorial duration, exact text/screens and skip/revisit behavior.
Project library	View/open, rename, delete.	Search, sort, folders and duplicates not required/confirmed.

Usability behavior

- User should be allowed to re-record or continue after an unsuitable-source warning.
- An import over 10 seconds should present source trimming rather than silently selecting the beginning.
- After source add/remove, ask whether to update the beat or preserve the current arrangement.
- Mode switching carries over compatible settings; do not introduce a history/version feature unless separately approved.
- Autosaving is required, but do not promise a specific autosave interval until implementation is selected and validated.
- The initial interface should make offline operation and user-controlled file selection clear without implying cloud processing.

11. Privacy, permissions, and platform behavior

Privacy and offline requirements

- All core functions work without an internet connection: recording, analysis, transformations, beat generation, editing, saving, import, and export.
- Project data remains on the device unless the user explicitly exports or shares an output.
- No account system, online analysis endpoint, cloud storage, telemetry service, or external AI API is part of the confirmed scope.
- The app should request microphone permission only for recording and explain the purpose when needed. Denial must be handled without crashing.
- Use user-mediated local file picking for importing; do not assume broad access to the user's audio library.
- Shared/exported files are intentionally user-directed. An Android share target may be an external app, but the app itself has no online sharing service.

Platform requirements

Setting	Decision/status
Minimum supported OS	Android 12+ (API 31 minimum) - confirmed.
Target SDK	OPEN; decide at implementation and re-check Play rules before release.

Setting	Decision/status
Compile SDK	OPEN; separate from target SDK and minimum OS.
Android Studio / Gradle / Kotlin plugin versions	OPEN; select a compatible stable toolchain during project setup.
Orientation	Portrait + landscape adaptive layout - confirmed.
Foreground playback	Stops when the app leaves the foreground or device is locked - confirmed.
Application ID / package namespace	OPEN; do not invent.
App name and icon	OPEN; do not invent.

Google Play readiness note

As of this blueprint date, Google Play's current target API policy says that starting 31 August 2026, new apps and app updates generally need to target Android 16 (API level 36) or higher. This is release context, not a locked target-SDK decision. Re-check the policy at implementation and before submission because the requirement can change [7].

12. Quality attributes, risks, and mitigations

Risk / constraint	Why it matters	Mitigation / validation
Not every input is musical	The only content is source-derived audio; some objects have little tonal/bass information.	Test varied recordings; provide several transforms; let users choose to continue after audio-quality warnings; do not promise every source becomes a polished beat.
Advanced DSP quality	Granular, pitch-shift and stretching algorithms can sound poor or be CPU-heavy.	Implement effects in increments; compare sound quality on real devices; profile; document which effects pass acceptance.
MP3 export implementation	Android device encoder availability may vary; a bundled encoder adds compatibility and licensing decisions.	Select and validate a local encoder; test emitted MP3 files on independent decoders; ensure export works offline.
Import codec diversity	WAV/MP3/M4A-AAC and “other practical formats” may not all decode consistently across phones.	Define the actually supported list after decoder/device verification; detect unsupported sources and give a clear message.
Real-time edit limits	Not every effect can be recalculated seamlessly while playing.	Apply only feasible simple controls live; stop/restart for edits that require regeneration, as confirmed.
Configurable grid/output limits	Unlimited dimensions/output length can create resource pressure.	Set safe bounds after testing; keep their values OPEN until validated.
Multi-source edit side effects	Changing source material can conflict with an existing arrangement.	Ask user whether to update or preserve; implement and test both paths after behavior is designed.
Autosave/data integrity	Autosave without backup/version history needs careful persistence handling.	Use atomic or recoverable writes if chosen; test app interruptions. Specific recovery strategy remains open.
Device/audio variability	Microphone capture and audio buffers vary across devices.	Run physical-device testing across supported Android 12+ devices; handle setup failures gracefully.
Changing Play requirements	Target API expectations can change before release.	Verify Google Play’s current policy when setting target SDK and again before submission.

Quality attributes to verify (no unsupported numeric targets)

- Functional correctness: actions behave as specified and saved projects are reopenable.
- Responsiveness: UI remains usable while analysis/render work runs; measure before setting numerical performance targets.
- Audio correctness: no clipping/obvious broken buffers in tested exports; exact loudness/quality target is open.
- Privacy: offline workflows succeed in airplane mode, and no source is uploaded by the app.
- Reliability: graceful handling of permission denial, unsupported audio, failed import, insufficient storage, and interrupted exports.
- Maintainability: keep UI, source management, DSP, beat sequencing, project persistence, and export as separable concerns.

13. Verification and acceptance checklist

These checks describe acceptance intent. Numerical performance thresholds and a final device matrix remain to be agreed. Run microphone/audio tests on real Android hardware; emulator support alone is not sufficient.

Test group	Pass condition
Input and permissions	Permission allow/deny; record and replay; countdown/warning; stop at 10 seconds; recover from recording initialization failure.
Long import	Import a file over 10 seconds; select a segment no longer than 10 seconds; verify only the selected segment is analyzed/used.
Formats	Test WAV, MP3, M4A/AAC and each additional format the implementation claims to support. Unsupported format handling must be clear.
Sound DNA	Each of six selected measurements is available after analysis; descriptions and visualizations appear; no extra metrics are treated as required.
Audio quality warning	Simulate or capture unsuitable inputs; user can re-record or continue; exact thresholds are tested only after chosen.
Mode discovery	Five modes are reachable; initial suggestion count adapts; remaining modes can be generated on demand.
Preview sequence	Transformed source preview appears before selection; beat-loop preview follows mode selection and precedes editor.
Transformation coverage	Each of the ten selected effects can be demonstrated on a source; results remain derived from source material.
No external sounds	Verify no bundled sample library or independent synthesized instrument is used by the beat generator.
Beat editing	All nine customization categories are usable; grid step count/loop length can be changed; loop can exceed 10 seconds.
Tempo behavior	Repeating-pulse case attempts estimation; no-pulse case gets editable default BPM.
Pad assignment	Pads auto-populate from available material; count is chosen based on segments; user can change count and reassign sounds.
Mode switching	Switching mode changes transformation while preserving compatible settings.
Multi-source	One-source and multi-source projects; primary-source recommendation and user override; add/remove prompt offers update vs preserve.
Projects	Autosave edits; close/reopen and continue; finished render remains; library view/open/rename/delete work.
Exports	Mix exports WAV and MP3; individual layers export WAV only; save destination and Android sharing both work offline.
Offline	Airplane-mode end-to-end test for record, analyze, transform, edit, save, import from device, export and share UI invocation.
Lifecycle	Playback stops when app leaves foreground and when phone locks.
Orientation	Portrait and landscape layouts rearrange without losing project/editor state.
Distribution	Install and smoke-test signed APK on selected devices; if publishing, validate current Play requirements and app identity.

14. Implementation roadmap

This is a dependency-ordered sequence, not a delivery schedule. No duration estimates have been inferred.

Phase	Deliverables / exit focus
Phase 0 - Project setup	Create Android Studio/Kotlin/Compose project; decide package/application ID and compatible build toolchain; make target/compile SDK choices when implementation begins.
Phase 1 - Source input and local persistence	Implement microphone capture, 10-second stop behavior, local file picking, trimming for longer imports, PCM/file representation, and local project skeleton.
Phase 2 - Playback and visuals	Implement source preview, foreground lifecycle stop behavior, waveform/frequency views, reactive visual layer, and orientation adaptation.
Phase 3 - Sound DNA	Implement the six metrics, intuitive descriptors, quality-warning flow, and source-based suggestion/recommendation foundations.
Phase 4 - DSP effects and modes	Implement and test the ten transformations; expose the five modes; create transformed-sound previews with adaptive initial count and on-demand remaining modes.
Phase 5 - Loop generator and editor	Generate a selected-mode loop preview; create grid scheduling, the nine customization controls, loop rendering, and compatible live-vs-regenerate edit behavior.
Phase 6 - Pads and multi-source projects	Add adaptive pad population/reassignment; support multiple source inputs; primary source recommendation/override; source add/remove prompt.
Phase 7 - Save, export, share	Autosave editable projects and renders; project library operations; WAV/MP3 mixed output; WAV layer output; device save/share; validate offline MP3 encoding.
Phase 8 - Stabilization	Test all functional requirements on real Android 12+ phones; validate codecs, storage edge cases, resource limits, playback lifecycle, and orientation.
Phase 9 - Distribution preparation	Create project demo APK; decide product identity; re-check Google Play policies; produce release signing and listing assets if public release proceeds.

Suggested milestone gates

- Gate A: microphone capture/import works and sources obey the 10-second rule.
- Gate B: one source can be analyzed and transformed locally across the selected modes/effects.
- Gate C: a selected transformation can produce and play a repeating loop in the editor.
- Gate D: pads and the sequencer operate on the same source-derived material.
- Gate E: project autosave/reopen and all specified export paths work offline.
- Gate F: acceptance checklist passes on real Android devices; distribution settings are reviewed.

15. Open decisions register

These items were not decided in the conversation. They remain intentionally open in this blueprint. They are not implicit requirements and should not be filled with arbitrary defaults.

ID	Area	Open item
OD-01	Product identity	Final app name, icon, package/application ID/namespace.
OD-02	Android build settings	Target SDK and compile SDK; exact Android Studio/Gradle/Kotlin plugin/dependency versions.
OD-03	DSP format	Internal sample rate, channel count, bit depth, PCM normalization and conversion rules.
OD-04	Audio import support	Exact supported file types beyond the requested WAV/MP3/M4A-AAC goal; decoder path and device compatibility.
OD-05	MP3 export	Offline encoder implementation, dependency/license, quality settings, and compatibility verification.
OD-06	Sound DNA algorithms	Windowing, calculation details, label vocabulary, thresholds, and mapping from metrics to suggestions.
OD-07	Quality detection	Unsuitable-input thresholds and exact warning rules.
OD-08	Preview generation	Adaptive initial preview-count logic, criteria for suggestions and preview ordering.
OD-09	Primary-source recommendation	Scoring/ranking method; how confidence or ties are handled.
OD-10	Tempo	Default BPM, allowed range, pulse-detection method and confidence threshold.
OD-11	Grid/loop bounds	Minimum/maximum step count, available loop durations, max output duration, maximum layers/events.
OD-12	Swing / timing	Exact timing/swing behavior, range and interaction with the configurable grid.
OD-13	Pads	Default count, min/max count, layout, simultaneous trigger behavior, and pad source assignment in multi-source projects.
OD-14	Multi-source scale	Maximum source count, per-source editing controls, and exact update/preserve implementation when sources change.
OD-15	DSP implementation	Algorithm/library choices for all ten effects; latency/performance acceptance thresholds.
OD-16	Storage schema	Exact project file format, metadata schema, autosave trigger/interval and interruption recovery strategy.
OD-17	Visual design	Palette, typography, actual labels, mode-preview cards, navigation/screen layouts, and animation details.
OD-18	Onboarding	Walkthrough copy, tutorial length, exact sequence and skip/revisit behavior.
OD-19	Distribution	Signing approach, release artifacts and current Google Play submission requirements at release.
OD-20	Unanswered pad question	Whether simultaneous sound-pad playback should be single-trigger, overlapping, or user-selectable. It was not answered before blueprint creation.

Decision discipline

Do not treat open items as defaults. Resolve them only when implementation or design requires the choice, and record the resulting decision in a later revision of this blueprint.

16. Release-readiness checklist and references

Project-demo readiness

- App installs and launches on selected Android 12+ phones.
- Microphone permissions, recording countdown/stop, local import/trim, and replay work.
- All six Sound DNA measurements and all five modes are reachable.
- All selected transformations are available and demonstrably derived from source audio.
- Loop editing, pads, autosave, project reopen, rename/delete and specified exports work.
- Airplane-mode test passes for all core workflows.
- No product name or app identifier is represented as final until chosen.

Potential Google Play readiness

- Choose target SDK and compile SDK during implementation, independently of the Android 12 minimum.
- Confirm current target API requirement immediately before submission. The current policy page states that new apps and updates must target Android 16/API 36 or higher from 31 August 2026 [7].
- Choose final application ID, app name, icon, signing configuration and store listing assets before submission.
- Test the release build on real devices and verify privacy/disclosure text matches actual behavior.
- Publication is a future possibility, not an approval or a confirmed launch date.

Official technical references

[1] Android Developers - Kotlin-first approach - https://developer.android.com/kotlin/first

[2] Android Developers - Develop UI / Jetpack Compose - https://developer.android.com/develop/ui

[3] Android Developers - AudioRecord.Builder reference - https://developer.android.com/reference/android/media/AudioRecord.Builder

[4] Android Developers - AudioTrack reference - https://developer.android.com/reference/android/media/AudioTrack

[5] Android Developers - Access documents and other files (Storage Access Framework) - https://developer.android.com/training/data-storage/shared/documents-files

[6] Android Developers - Access media files from shared storage - https://developer.android.com/training/data-storage/shared/media

[7] Google Play Console Help - Target API level requirements for Google Play apps - https://support.google.com/googleplay/android-developer/answer/11926878

Revision record

Version	Date	Change
1.0	13 Oct 2026	First blueprint assembled from confirmed ideation decisions. Open questions are explicitly registered; implementation has not yet been verified.